

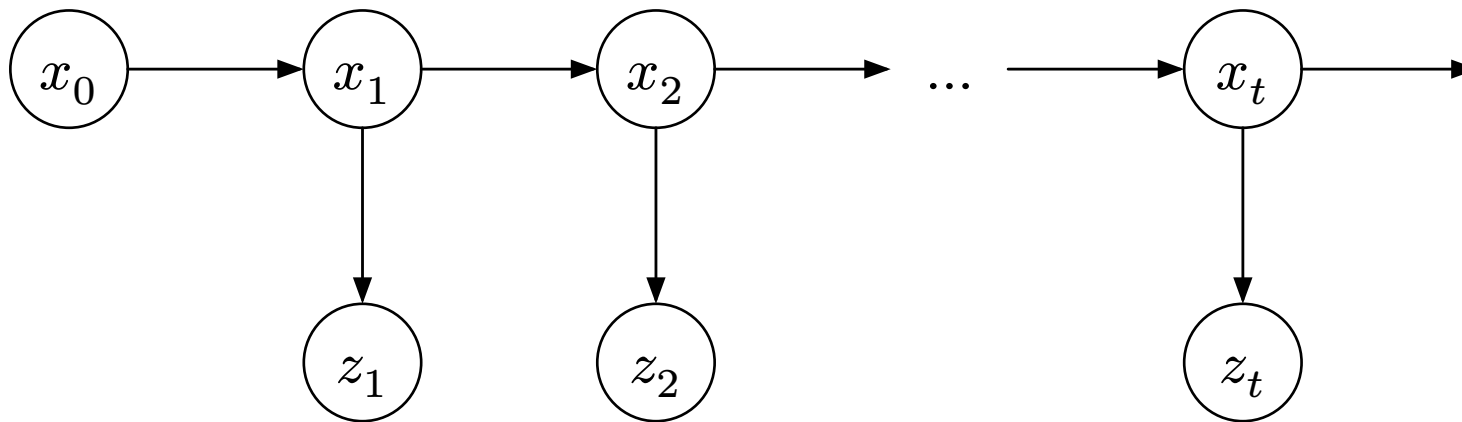
Particle Filters

Based on *A tutorial on particle filters* (Speekenbrink, 2016)



State-space models (SSMs)

Observable $z_{1:t}$ through a time-series of latent states $x_{0:t}$.



Want to estimate x_t given observations $z_{1:t}$.

Core assumptions

1. Each observation z_t only depend on current state x_t

$$p(z_{1:T}|x_{0:T}) = \prod_{t=1}^T p(z_t|x_t).$$

2. Latent states evolve according to a first-order Markov process

$$p(x_{0:T}) = p(x_0) \prod_{t=1}^T p(x_t|x_{t-1}).$$

Linear and Gaussian example

$$\mathbf{x}_t = \mathbf{A}\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

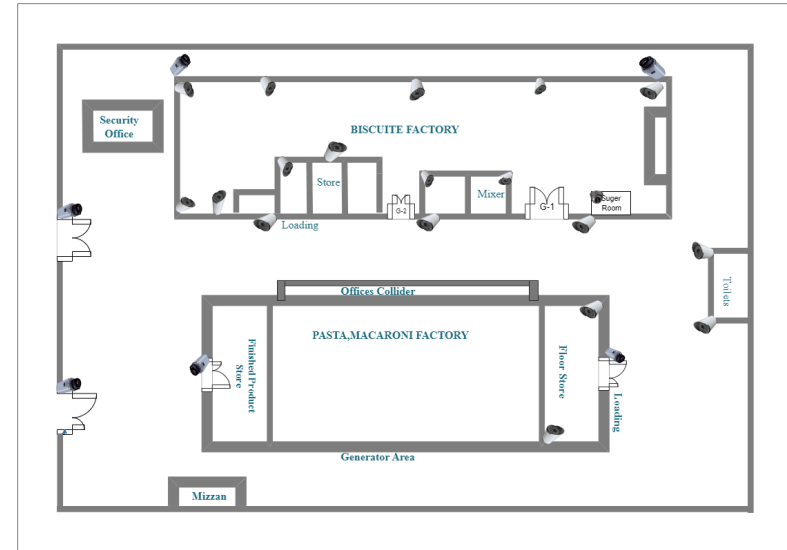
$$\mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

In this case, the posterior $p(\mathbf{x}_t | \mathbf{z}_{1:t})$ is Gaussian and computed analytically in closed form via recursive updates (Kalman filter).

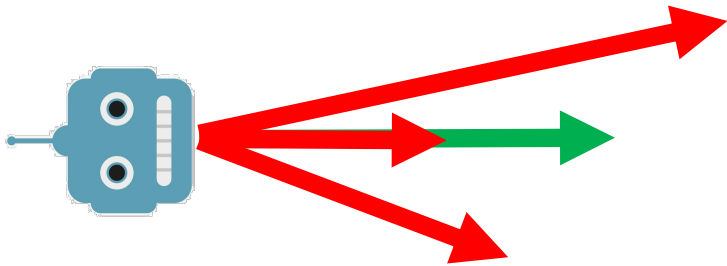
But...what about non-linear and/or non-Gaussian SSMs?

Application: Robot localization problem

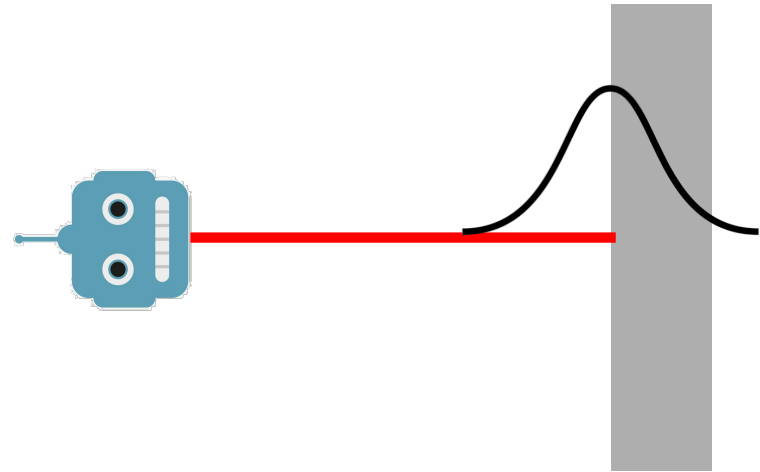
Estimate the robot position (state) given movement commands and sensor measurements (observations).



The robot has two main flaws:



(a) Movement can be noisy.



(b) Sensor measurements can be noisy.

Note that robot knows what the map looks like, but not where it is on the map.

Let us formalize the

Up to time t :

- Let $x_{0:t}$ be the states (position)
- Let $u_{0:t}$ be the control input (movement commands)
- Let $z_{1:t}$ be the observations (sensor measurements)

For functions f and g , we have

$$x_t = f(x_{t-1}, u_t) + \eta_t$$

and

$$z_t = g(x_t) + \varepsilon_t$$

where η_t and ε_t are non-Gaussian noise distributions.

Possible approaches

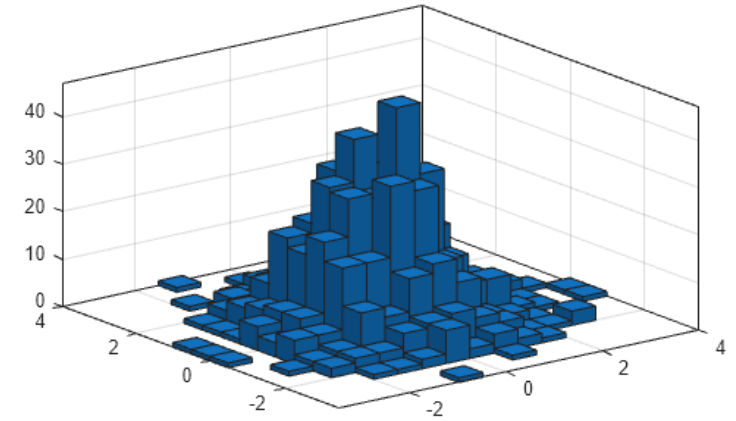
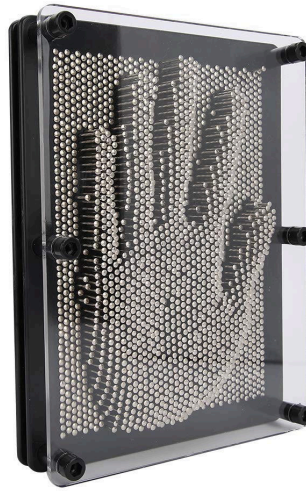
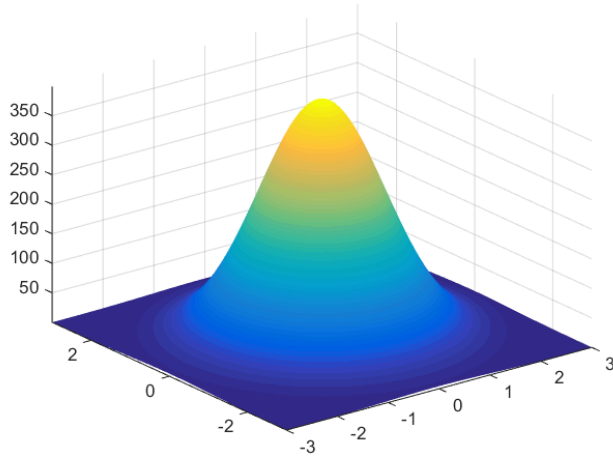
Extended Kalman filter Does not work well when the non-linearities are strong, and assumes Gaussian noise.

Ensemble Kalman filter Can handle non-linearities better, but still assumes Gaussian noise.

Particle filter Can handle non-linearities and non-Gaussian noise.

Theory

From Gaussian distributions to a cloud of samples



Particles: Probability as a Histogram in Motion

Each particle is a tiny “grain of probability”, and the collection as a whole shifts and reshapes over time. When new evidence arrives, we adjust the weights of these grains so that the cloud of particles continues to approximate the true probability distribution. (Dhayalkar, 2025)

Particles: More formally

N particles, each with an associated weight

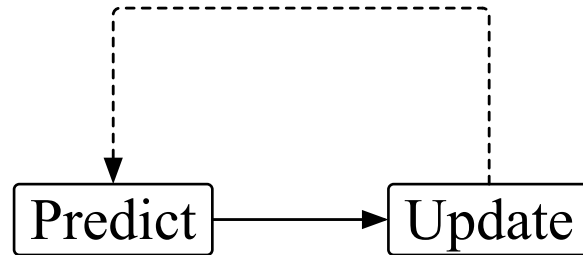
$$\left\{ x_k^{(i)}, w_k^{(i)} \right\}_{i=1}^N, \quad w_k^{(i)} \geq 0, \quad \sum_{i=1}^N w_k^{(i)} = 1$$

Approximate the posterior as

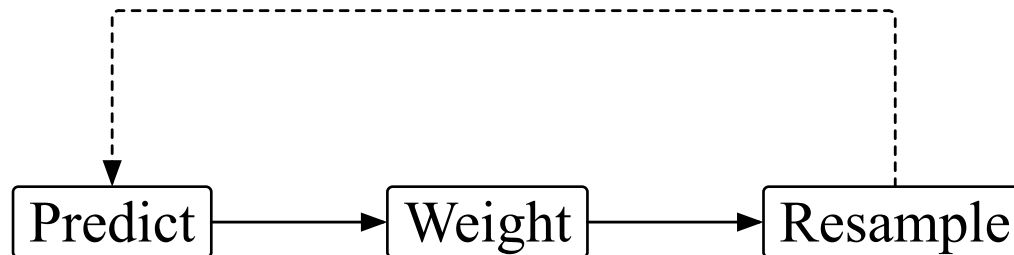
$$p(x_k | z_{1:k}) \approx \sum_{i=1}^N w_k^{(i)} \delta(x_k - x_k^{(i)}), \quad \text{Mean } \hat{x}_k = \sum_{i=1}^N w_k^{(i)} x_k^{(i)}$$

How to evolve particle filter over time?

Kalman filter:



Particle filter:



See [this animation](#).

Step 1: Predict - *Carry our beliefs forward*

- At time $k - 1$ we have particles $\left\{ x_{k-1}^{(i)}, w_{k-1}^{(i)} \right\}_{i=1}^N$ approximating $p(x_{k-1} | z_{1:k-1})$.
- Propagate each particle forward by our state transition model

$$x_k^{(i)} = f\left(x_{k-1}^{(i)}, u_k\right) + \eta_k$$

- Note that observation at time k is not used!

Step 2: Weight - *Incorporate new observation*

- Adjust weights so we approximate “posterior” $p(x_k | z_{1:k})$ instead of “prior” $p(x_k | z_{1:k-1})$.
- Particles consistent with observations get higher weights.
- This will require some work...

Importance sampling

Approximate $p(x)$ using samples from a different distribution $q(x)$

$$\mathbb{E}_p[g(x)] = \int g(x)p(x) \, dx = \int g(x) \frac{p(x)}{q(x)} q(x) \, dx \approx \frac{1}{N} \sum_{i=1}^N g(x^{(i)}) w^{(i)}$$

where $x^{(i)} \sim q(x)$ and $w^{(i)} = \frac{p(x^{(i)})}{q(x^{(i)})}$.

Sequential importance sampling for SSMs

We can apply importance sampling sequentially at each time step. Something like

$$w_k^{(i)} = \frac{p(x_{0:k} | z_{1:k})}{q_k(x_{0:k})}$$

However, this is not practical because the weights depend on all previous states.

Sequential importance sampling for SSMs

Using the SSM assumptions, we get

$$p(x_{0:k}|z_{1:k}) = \frac{p(z_k|x_k)p(x_k|x_{k-1})}{p(z_k|z_{1:k-1})} \cdot p(x_{0:k-1}|z_{1:k-1})$$

and we can choose the proposal distribution so that

$$q_k(x_{0:k}) = q_k(x_k|x_{0:k-1}) \cdot q_{k-1}(x_{0:k-1}).$$

Then, the weights can be computed recursively as

$$w_k^{(i)} = \frac{p(z_k|x_k^{(i)})p(x_k^{(i)}|x_{k-1}^{(i)})}{p(z_k|z_{1:k-1})q_k(z_k|x_{k-1}^{(i)})} \cdot w_{k-1}^{(i)}$$

Sequential importance sampling for SSMs

A common proposal distribution is the Bootstrap filter $q_k(x_k|x_{k-1}) = p(x_k|x_{k-1})$. Can also ignore $p(z_k|z_{1:k-1})$ since it is constant across particles. The simplified weights are

$$\tilde{w}_k^{(i)} = p(z_k|x_k^{(i)}) \cdot w_{k-1}^{(i)}$$

which we can normalize to get the final weights

$$w_k^{(i)} = \frac{\tilde{w}_k^{(i)}}{\sum_{j=1}^N \tilde{w}_k^{(j)}}$$

The set $\left\{x_k^{(i)}, w_k^{(i)}\right\}_{i=1}^N$ now approximates $p(x_k|z_{1:k})$.

Weight degeneracy

Over time, a few particles may carry most of the total weight.

Effective sample size:

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N \left(w_k^{(i)}\right)^2}$$

When

- $N_{\text{eff}} = N$: All weights are equal (full diversity)
- $N_{\text{eff}} = 1$: One particle has all the weight (complete degeneracy)

Step 3: Resampling - *Focusing on what matters*

If $N_{\text{eff}}/N \leq c$:

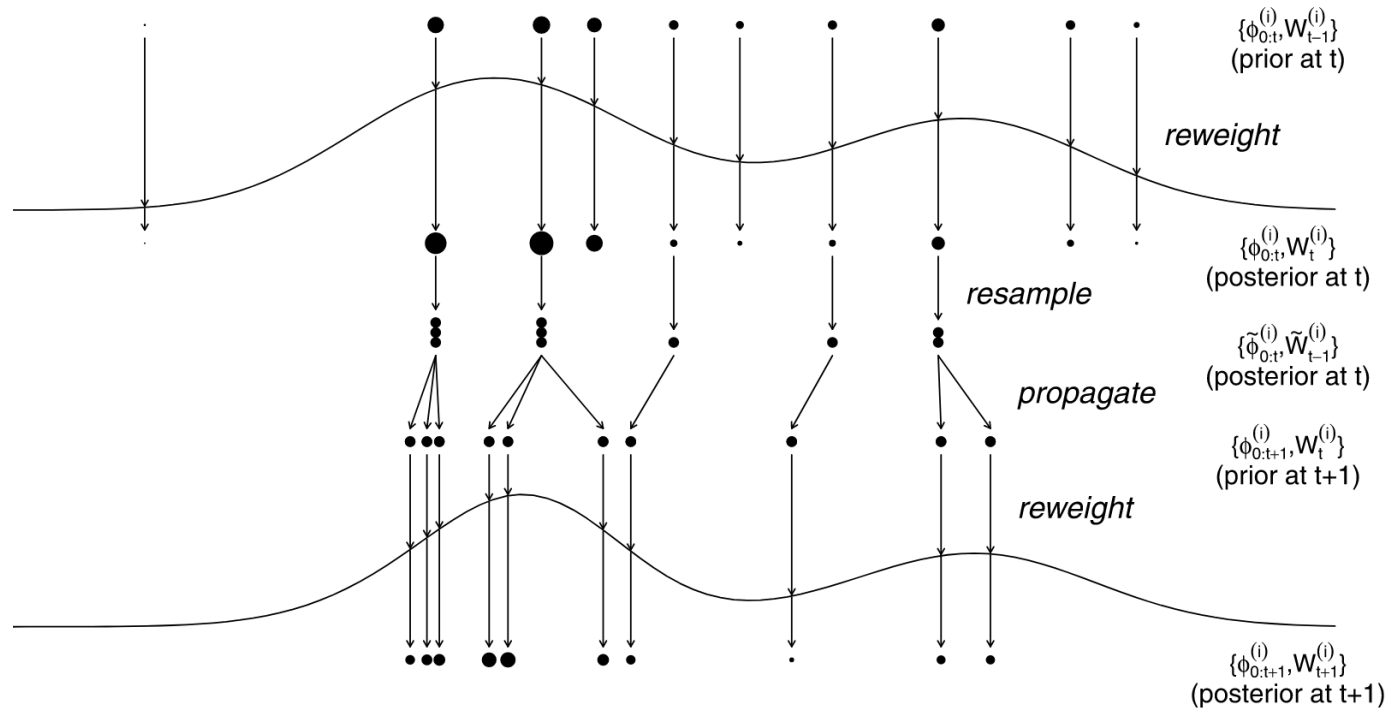
- Resample locations $x_k^{(i)}$ according to weights: $P(i' = i) = w_k^{(i)}$.
- After resampling, set $w_k^{(i)} = 1/N$ for all particles.

The resampled set now is an “unweighted” approximation of the posterior.

Sample impoverishment: An issue with resampling is that we reduce the number of unique locations present in set of particles.

Full algorithm

1. **Predict:** Move particles according to state transition model.
2. **Weight:** Correct estimate by comparing prediction with observation.
3. **Resample:** If necessary, resample/regenerate particles to prevent degeneracy.



That was it. Thank you!

Bibliography

- Dhayalkar, S.R., 2025. Particle Filter Made Simple: A Step-by-Step Beginner -friendly Guide [WWW Document].. <https://doi.org/10.48550/arXiv.2511.01281>
- Speekenbrink, M., 2016. A tutorial on particle filters. *Journal of Mathematical Psychology* 73, 140–152.. <https://doi.org/10.1016/j.jmp.2016.05.006>